UNIT-II

Requirements Engineering and System Modeling

CHAPTER 4,5,6

Contents

- Functional and non-functional requirements
- The software requirements specification document(SRS)
- Requirements specification
- Requirements engineering processes
- Requirements elicitation and analysis
- Requirements validation
- Requirements management

Outcomes

- understand the concepts of user and system requirements and why these requirements should be written in different ways;
- understand the differences between functional and nonfunctional software requirements;
- understand how requirements may be organized in a software requirements document;
- understand the principal requirements engineering activities of elicitation, analysis and validation, and the relationships between these activities;
- understand why requirements management is necessary and how it supports other requirements engineering activities

Requirements Definition

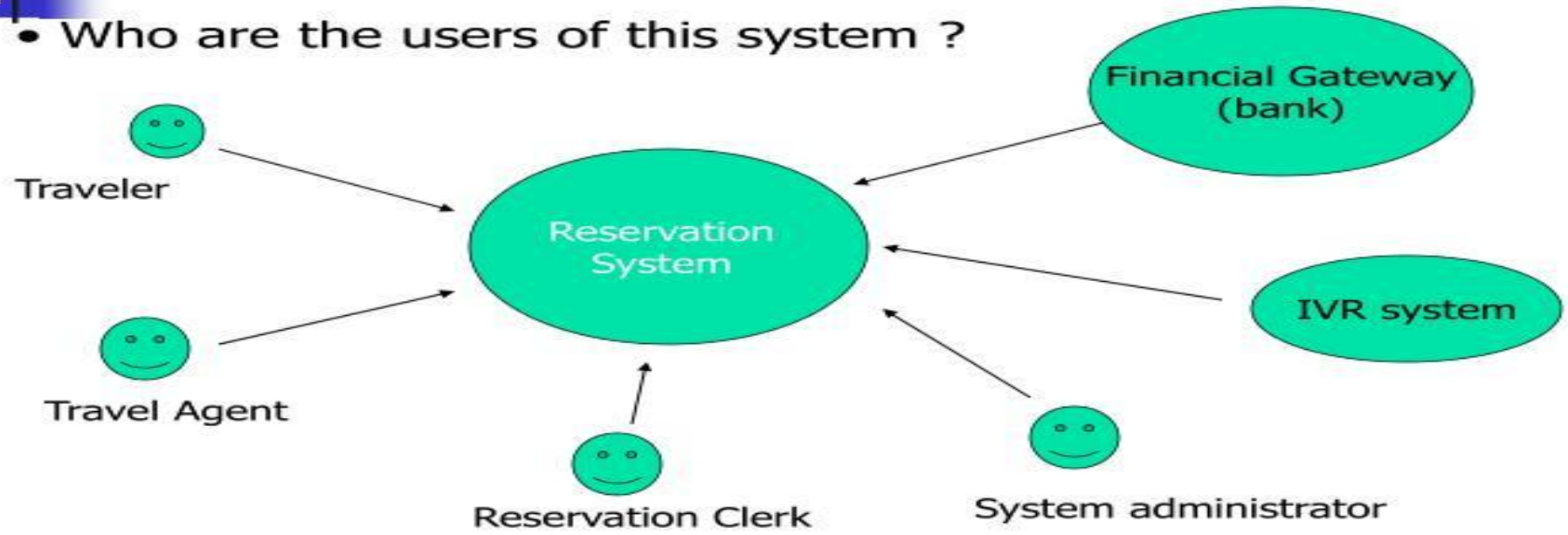
- It may range from a high-level abstract statement of a service or of a system constraint to a detailed mathematical functional specification
- The software requirements are description of features and functionalities of the target system
- Requirements convey the expectations of users from the software product.

Requirements engineering

- The process of establishing the services that the customer requires from a system and the constraints under which it operates and is developed

Railway reservation system

- Who are the users of this system ?



Product Perspective:

Before the automation, the system suffered from the following **DRAWBACKS:**

- Ø The existing system is highly manual involving a lot of paper work and calculation and therefore may be erroneous. This has lead to inconsistency and inaccuracy in the maintenance of data.
- Ø The data, which is stored on the paper only, may be lost, stolen or destroyed due to natural calamity like fire and water.
- Ø The existing system is sluggish and consumes a lot of time causing inconvenience to customers and the airlines staff.
- Ø Due to manual nature, it is difficult to update, delete, add or view the data.
- Ø Since the number of passengers have drastically increased therefore maintaining and retrieving detailed record of passenger is extremely difficult.
- Ø An railways has many offices around the world, an absence of a link between these offices lead to lack of coordination and communication.

Hence the railways reservation system is proposed with the following

- Ø The computerization of the reservation system will reduce a lot of paperwork and hence the load on the airline administrative staff.
- Ø The machine performs all calculations. Hence chances of error are nil.
- Ø The passenger, reservation, cancellation list can easily be retrieved and any required addition, deletion or updation can be performed.
- Ø The system provides for user-ID validation, hence unauthorized access is prevented.

Types of requirement

- **User requirements**

  Statements in natural language plus diagrams of the services the system provides and its operational constraints. Written forcustomers.

- **System requirements**

  A structured document setting out detailed descriptions of the system’s functions, services and operational constraints. Defines what should be implemented so may be part of a contract between client and contractor.

User requirements

- **User requirements** are generally given by the **user** and used as the primary input for creating system **requirements.**
- **User Requirements** describe how **user** expects and how they will interact with the product.
- Examples:

 search for songs by artist, album title and/or genre.

 search for books by author, title and/or ISBN number.

 Site visitors should be able to search the database using relevant keywords to identify items of interest.

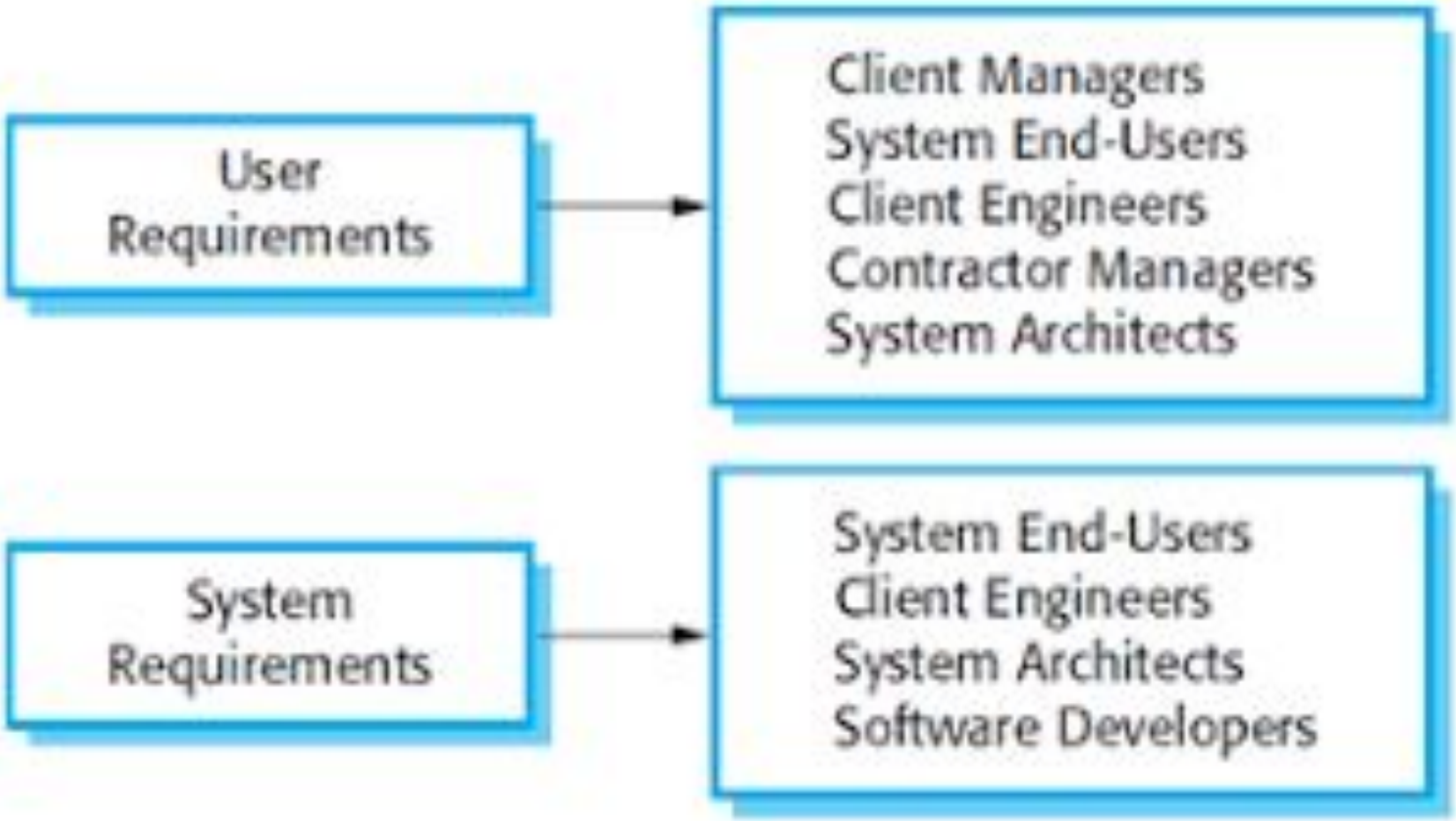
System requirements

- **System requirements** are all of the **requirements** at the **system** level that describe the functions which the **system** as a whole should fulfill to satisfy the stakeholder needs and **requirements.**

Examples

- User requirement
  - We need to be able to spell check documents
- System requirement
  - The system needs to be able to spell check documents and provide autocorrect facilities. Their will be support for the following languages, English, French and German will plug in support for other languages

Readers of different types of requirements specification



Functional and non-functional requirements

- **Functional requirements**
- Statements of services the system should provide, how the system should react to particular inputs and how the system should behave in particular situations.
- May state what the system should not do.

- **Non-functional requirements**
- Constraints on the services or functions offered by the system
- such as timing constraints, constraints on the development process, standards, etc.
- Often apply to the system as a whole rather than individualfeatures or services.

Functional requirements of library management system

- The Librarian :

1. Add Article
2. Update Article
3. Delete Article
4. Inquiry Members
5. Check out Article

Functional requirements of library management system

- Member

1. Authentication
2. Search Article
3. Request Article
4. Check Account

Functional requirements

- The web application shall accept customer orders
- The web application shall be able to cash a sale
- The web application shall produce a receipt detailing a customers' purchase information and include name of customer, items purchased, cost of each item and total cost
- The web application shall be able to produce weekly, monthly and yearly reports about sales

Non functional requirements

- The web application shall be easy to use by all employees including sales representatives and managers
- The web application shall be available in several languages
- The web application shall allow several sales to be made at the same time without downgrading performance

Functional requirements

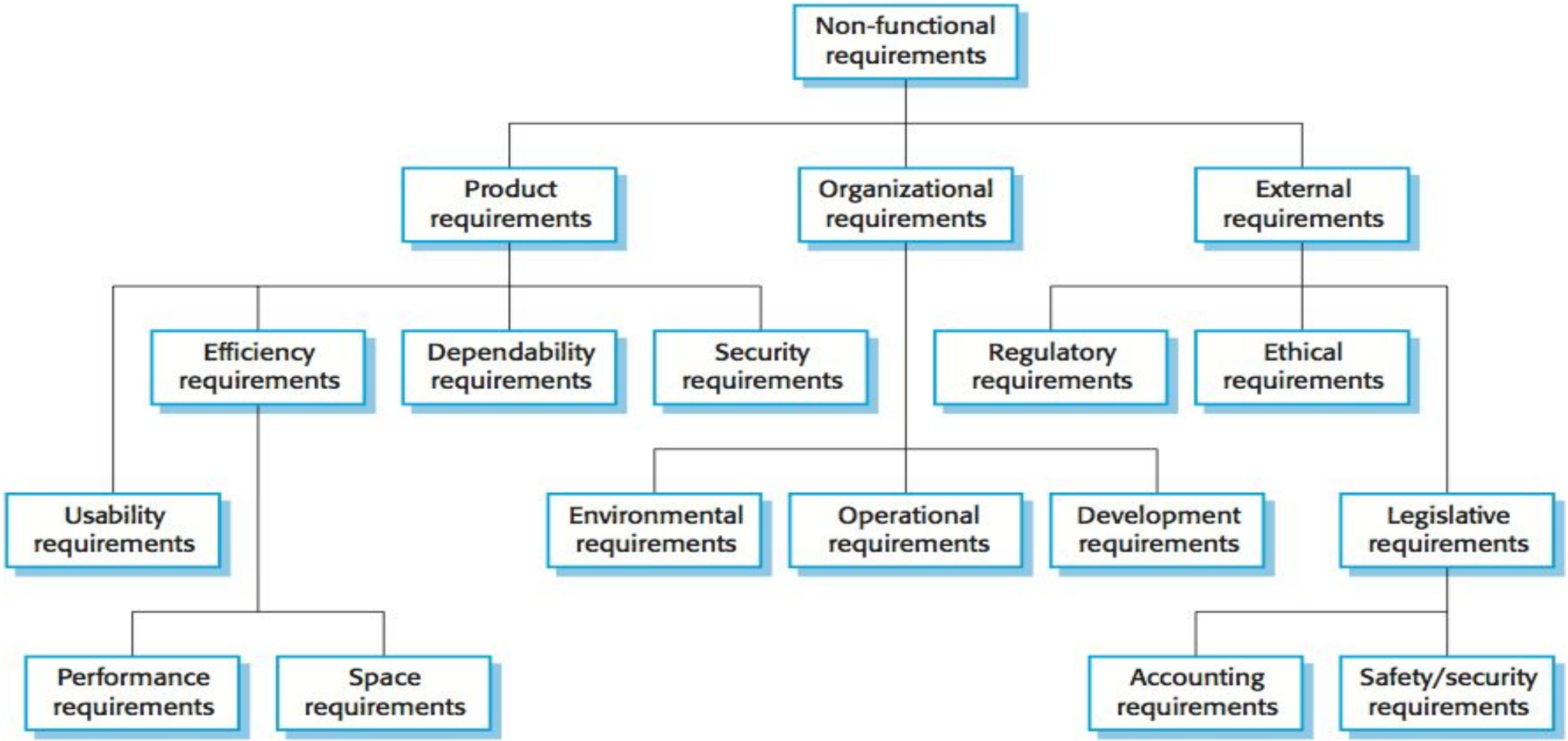
- Describe functionality or system services.
- Depend on the type of software, expected users and the type of system where the software is used.
- Functional user requirements may be high-level statements of what the system should do.
- Functional system requirements should describe the system services in detail

Requirements problems

- Complete
- Consistent
- Ambiguous

Non-functional requirements

- These define system properties and constraints e.g.reliability, response time, and storage requirements.
- Constraints are I/O device capability, system representations, etc.
- Process requirements may also be specified mandating a particular IDE, programming language or development method.
- Non-functional requirements may be more critical than functional requirements. If these are not met, the system may be useless.


Non-functional requirements
Product requirements
Organizational requirements
External requirements
Efficiency requirements
Dependability requirements
Security requirements
Regulatory requirements
Ethical requirements
Usability requirements
Environmental requirements
Operational requirements
Development requirements
Legislative requirements
Performance requirements
Space requirements
Accounting requirements
Safety/security requirements

Non-functional requirements implementation

- Non-functional requirements may affect the overall architecture of a system rather than the individual components.
- A single non-functional requirement, such as a security requirement, may generate a number of related functional requirements that define system services that are required.

Non-functional classifications

- ***Product requirements***
  - Requirements which specify that the delivered product must behave in a particular way e.g. execution speed, reliability, etc.
- ***Organizational requirements***
  - Requirements which are a consequence of organizational policies and procedures e.g. process standards used,implementation requirements, etc.
- ***External requirements***
  - Requirements which arise from factors which are external to the system and its development process, e.g. interoperability requirements, legislative requirements, etc.

Metrics for specifying non-functional requirements

Property	Measure
Speed	Processed transactions/second User/event response time Screen refresh time
Size	Mbytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

Domain requirements

- The system’s operational domain imposes requirements on the system.
- Domain requirements can be new functional requirements, constraints on existing requirements, or

define specific computations.

- If domain requirements are not satisfied, the system may be unworkable.

Domain requirements problems

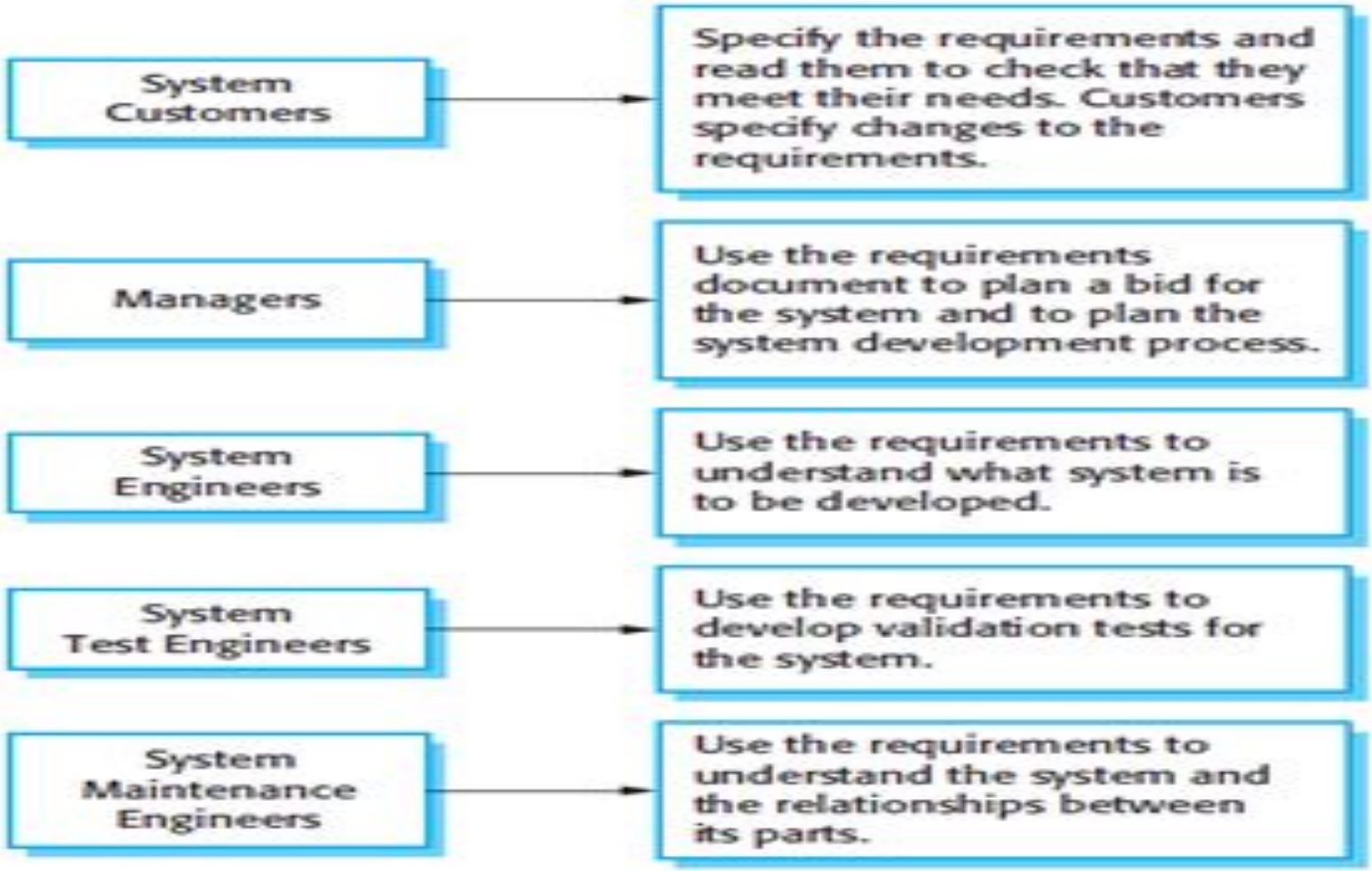
- Understandability
- Implicitness

The software requirements document

- The software requirements document is the official statement of what is required of the system developers.
- Can include both a definition of user requirements and a specification of the system requirements.
- It is NOT a design document. As far as possible, it should set of WHAT the system should do rather than HOW it should do it.

Agile methods and requirements

- Many agile methods argue that producing a requirements document is a waste of time as requirements change so quickly
- The document is therefore always out of date
- Methods such as XP use incremental requirements engineering and express requirements as ‘user stories’.
- This is practical for business systems but problematic for systems that require a lot of pre-delivery analysis (e.g.critical systems) or systems developed by several teams


System Customers
Specify the requirements and read them to check that they meet their needs. Customers specify changes to the requirements.
Managers
Use the requirements document to plan a bid for the system and to plan the system development process.
System Engineers
Use the requirements to understand what system is to be developed.
System Test Engineers
Use the requirements to develop validation tests for the system.
System Maintenance Engineers
Use the requirements to understand the system and the relationships between its parts.

The structure of a requirements document

Chapter	Description
Preface	This should define the expected readership of the document and describe its version history, including a rationale for the creation of a new version and a summary of the changes made in each version.
Introduction	This should describe the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organization commissioning the software.
Glossary	This should define the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader.
User requirements definition	Here, you describe the services provided for the user. The nonfunctional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified.
System architecture	This chapter should present a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules. Architectural components that are reused should be highlighted.

Structure of the SRS Document

Chapter	Description
System requirements specification	This should describe the functional and nonfunctional requirements in more detail. If necessary, further detail may also be added to the nonfunctional requirements. Interfaces to other systems may be defined.
System models	This might include graphical system models showing the relationships between the system components and the system and its environment. Examples of possible models are object models, data-flow models, or semantic data models.
System evolution	This should describe the fundamental assumptions on which the system is based, and any anticipated changes due to hardware evolution, changing user needs, and so on. This section is useful for system designers as it may help them avoid design decisions that would constrain likely future changes to the system.
Appendices	These should provide detailed, specific information that is related to the application being developed; for example, hardware and database descriptions. Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organization of the data used by the system and the relationships between data.
Index	Several indexes to the document may be included. As well as a normal alphabetic index, there may be an index of diagrams, an index of functions, and so on.

Requirements specification

- The process of writing down the user and system requirements in a requirements document
- User requirements have to be understandable by end users and customers who do not have a technical background
- System requirements are more detailed requirements and may include more technical information
- The requirements may be part of a contract for the system development
- It is therefore important that these are as complete as possible

Ways of writing a system requirements specification

Notation	Description
Natural language sentences	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Design description languages	This approach uses a language like a programming language, but with more abstract features to specify the requirements by defining an operational model of the system. This approach is now rarely used although it can be useful for interface specifications.
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract.

Natural language specification

- Requirements are written as natural language sentences supplemented by diagrams and tables.
- Used for writing requirements because it is expressive, intuitive and universal. This means that the requirements
- can be understood by users and customers

example

- ***Example**: "A system shall allow the users to register by entering their username and password, In order to get an access to the system".*
- When we say "a system", this word is very vague, we need to define what exactly the part of the system that will take care of this requirement.
- We may highlight the important keywords.
- Don't use abbreviations, and acronyms, have to add of what's called "Appendix". It defines all the abbreviations and acronyms in the specification and their relevant meaning.

Requirements and design

- In principle, requirements should state what the system should do and the design should describe how it does this.
- In practice, requirements and design are inseparable
- A system architecture may be designed to structure the requirements;
- The system may inter-operate with other systems that generate design requirements;
- The use of a specific architecture to satisfy non-functional requirements may be a domain requirement.
- This may be the consequence of a regulatory requirement.

Guidelines for writing requirements

- Create a standard format and use it for all requirements.
- Use language in a consistent way. Use shall for mandatory requirements, should for desirable requirements.
- Use text highlighting to identify key parts of therequirement.
- Avoid the use of computer jargon.
- Include an explanation (rationale) of why a requirement is necessary

Problems with natural language

- Lack of clarity
  - Precision is difficult without making the document difficult to read.
- Requirements confusion
  - Functional and non-functional requirements tend to be mixed-up.
- Requirements amalgamation
  - Several different requirements may be expressed together

Structured specifications

- An approach to writing requirements where the freedom of the requirements writer is limited and requirements are written in a standard way.
- This works well for some types of requirements e.g. requirements for embedded control system but is sometimes too rigid for writing business systemrequirements.
- It uses standard templates to specify the requirements

Example

Insulin Pump/Control Software/SRS/3.3.2	
Function	Compute insulin dose: Safe sugar level
Description	Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units
Inputs	Current sugar reading (r2), the previous two readings (r0 and r1)
Source	Current sugar reading from sensor. Other readings from memory.
Outputs	CompDose – the dose in insulin to be delivered
Destination	Main control loop
Action	CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered.
Requires	Two previous readings so that the rate of change of sugar level can be computed.
Pre-condition	The insulin reservoir contains at least the maximum allowed single dose of insulin
Post-condition	r0 is replaced by r1 then r1 is replaced by r2
Side-effects	None

Form-based specifications

- Definition of the function or entity.
- Description of inputs and where they come from.
- Description of outputs and where they go to.
- Information about the information needed for the computation and other entities used.
- Description of the action to be taken.
- Pre and post conditions (if appropriate).
- The side effects (if any) of the function

Tabular specification

- Used to supplement natural language
- Particularly useful when you have to define a number of possible alternative courses of action
- For example, the insulin pump systems bases its computations on the rate of change of blood sugar level and the tabular specification explains how to calculate
- the insulin requirement for different scenarios

Requirements engineering processes

- The processes used for RE vary widely depending on the application domain, the people involved and the organisation developing the requirements.
- However, there are a number of generic activities common to all processes

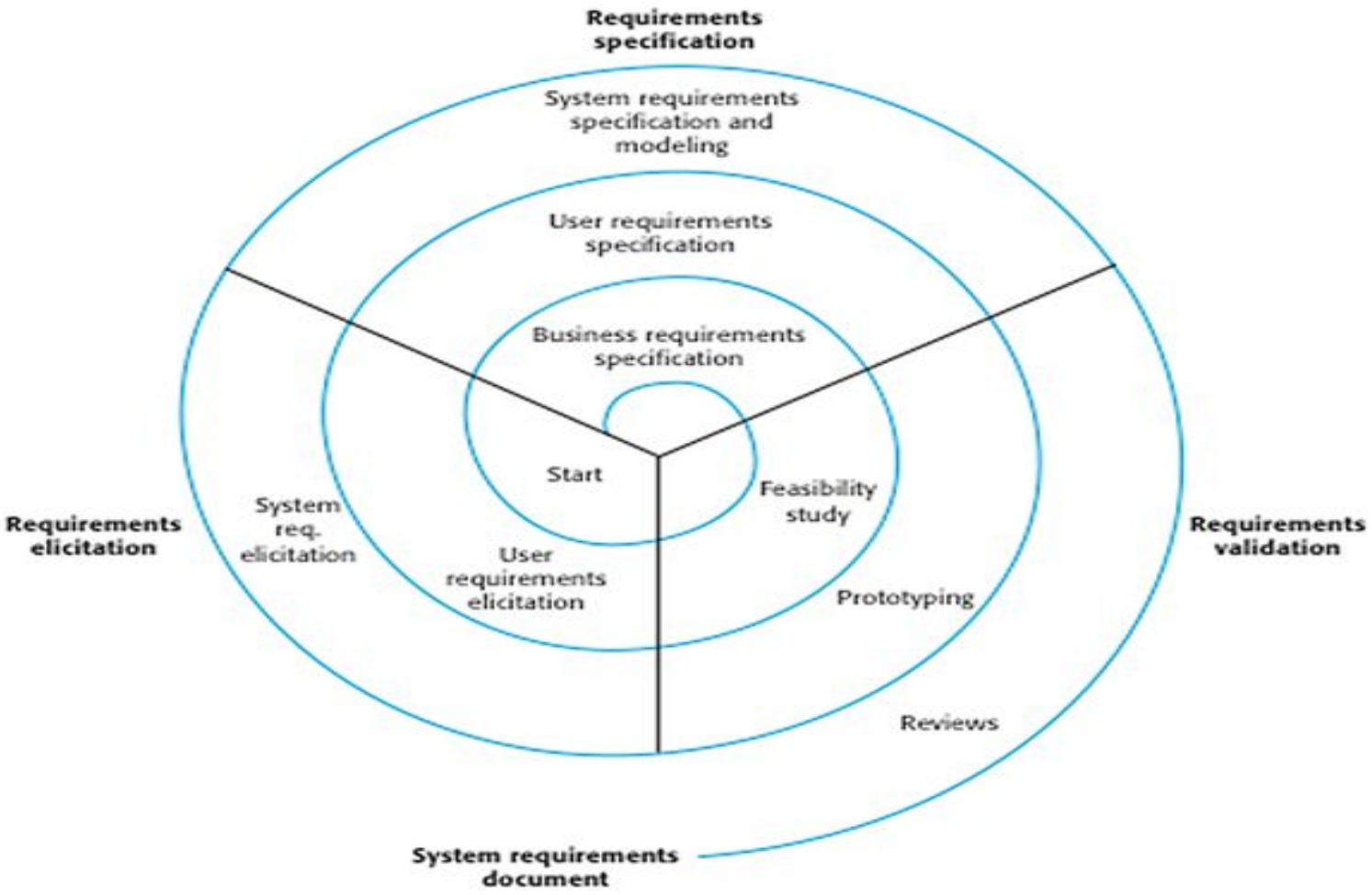
 Requirements elicitation;

 Requirements analysis;

 Requirements validation;

 Requirements management.

- In practice, RE is an iterative activity in which these processes are interleaved.


Requirements specification
System requirements specification and modeling
User requirements specification
Business requirements specification
Start
Feasibility study
Requirements elicitation
System req. elicitation
User requirements elicitation
Requirements validation
Prototyping
Reviews
System requirements document

Requirements elicitation and analysis

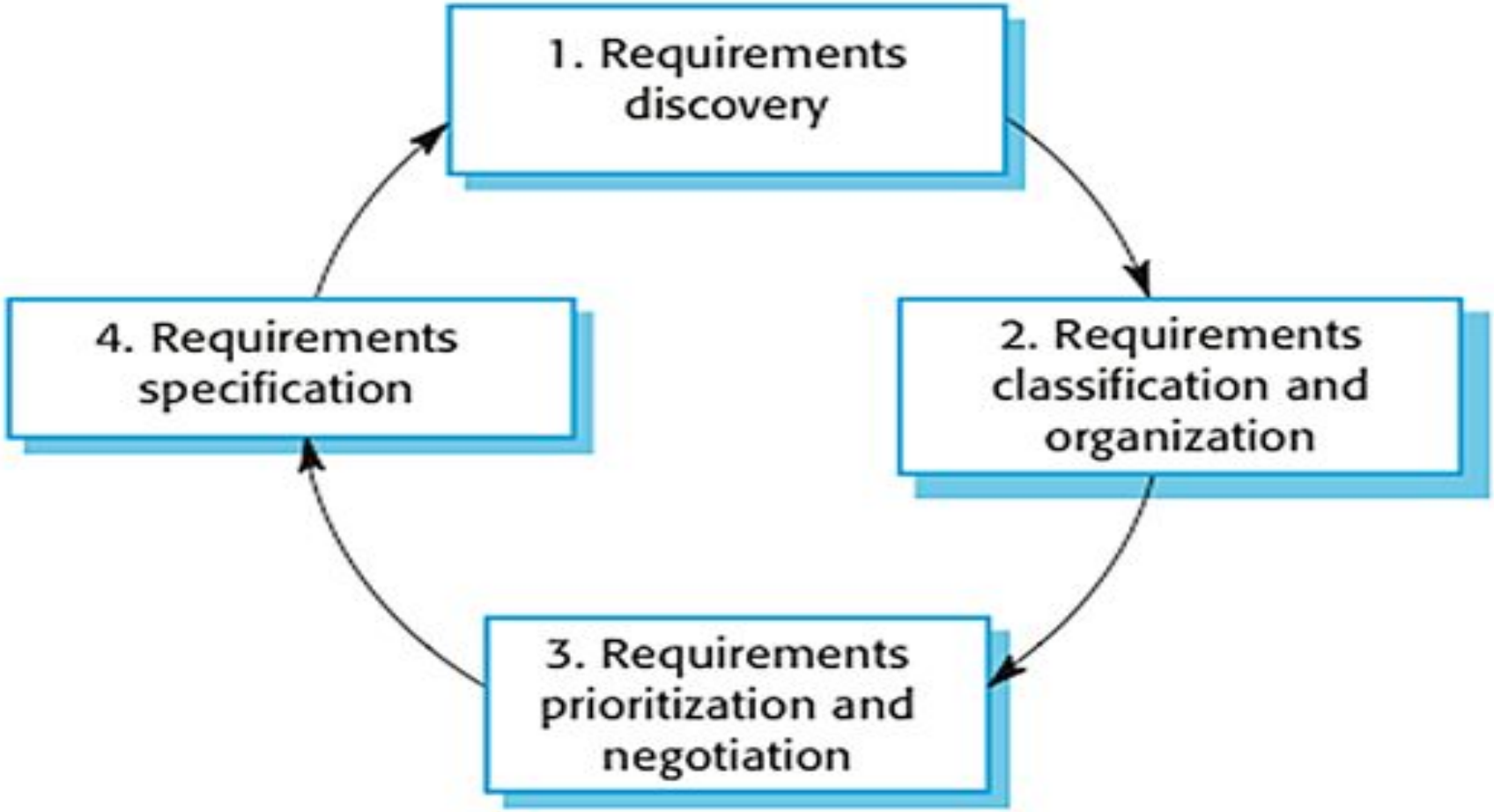
- Sometimes called requirements elicitation or requirements discovery.
- Involves technical staff working with customers to find out about the application domain, the services that the system should provide and the system's operational constraints.
- May involve end-users, managers, engineers involved in maintenance, domain experts, trade unions, etc. These are called stakeholders.

Problems of requirements analysis

- Stakeholders don’t know what they really want.
- Stakeholders express requirements in their own terms.
- Different stakeholders may have conflicting requirements.
- Organizational and political factors may influence the system requirements.
- The requirements change during the analysis process.
- New stakeholders may emerge and the business environment may change.

Requirements elicitation and analysis

- Software engineers work with a range of system stakeholders to find out about the application domain, the services that the system should provide, the required system performance, hardware constraints, other systems, etc.
- Stages include:
- Requirements discovery,
- Requirements classification and organization,
- Requirements prioritization and negotiation,
- Requirements specification


1. Requirements discovery
2. Requirements classification and organization
3. Requirements prioritization and negotiation
4. Requirements specification

Process activities

- **Requirements discovery**
- Interacting with stakeholders to discover their requirements. Domain requirements are also discovered at this stage.
- **Requirements classification and organization**
- Groups related requirements and organizes them into coherent clusters.
- **Prioritization and negotiation**
- Prioritizing requirements and resolving requirements conflicts.
- **Requirements specification**
- Requirements are documented and input into the next round of the spiral.

Requirements discovery

- The process of gathering information about the required and existing systems and distilling the user and system requirements from this information.
- Interaction is with system stakeholders from managers to external regulators.
- Systems normally have a range of stakeholders

Interviewing

- Formal or informal interviews with stakeholders are part of most RE processes.

 Types of interview

✔ Closed interviews based on pre-determined list of questions

✔ Open interviews where various issues are explored with stakeholders.

 Effective interviewing

✔ Be open-minded, avoid pre-conceived ideas about the requirements and are willing to listen to stakeholders.

✔ Prompt the interviewee to get discussions going using a springboard question, a requirements proposal, or by working together on a prototype system.

Interviews in practice

- Normally a mix of closed and open-ended interviewing.
- Interviews are good for getting an overall understanding of what stakeholders do and how they might interact with the system.
- Interviews are not good for understanding domain requirements
- Requirements engineers cannot understand specific domain terminology;
- Some domain knowledge is so familiar that people find it hard to articulate or think that it isn’t worth articulating.

Scenarios

- Scenarios are real-life examples of how a system can be used.
- They should include
- A description of the starting situation;
- A description of the normal flow of events;
- A description of what can go wrong;
- Information about other concurrent activities;
- A description of the state when the scenario finishes

Scenario for collecting medical history in MHCPMS

Initial assumption: The patient has seen a medical receptionist who has created a record in the system and collected the patient's personal information (name, address, age, etc.). A nurse is logged on to the system and is collecting medical history.

Normal: The nurse searches for the patient by family name. If there is more than one patient with the same surname, the given name (first name in English) and date of birth are used to identify the patient.

The nurse chooses the menu option to add medical history.

The nurse then follows a series of prompts from the system to enter information about consultations elsewhere on mental health problems (free text input), existing medical conditions (nurse selects conditions from menu), medication currently taken (selected from menu), allergies (free text), and home life (form).

Scenario for collecting medical history in MHCPMS

What can go wrong: The patient's record does not exist or cannot be found. The nurse should create a new record and record personal information.

Patient conditions or medication are not entered in the menu. The nurse should choose the 'other' option and enter free text describing the condition/medication.

Patient cannot/will not provide information on medical history. The nurse should enter free text recording the patient's inability/unwillingness to provide information. The system should print the standard exclusion form stating that the lack of information may mean that treatment will be limited or delayed. This should be signed and handed to the patient.

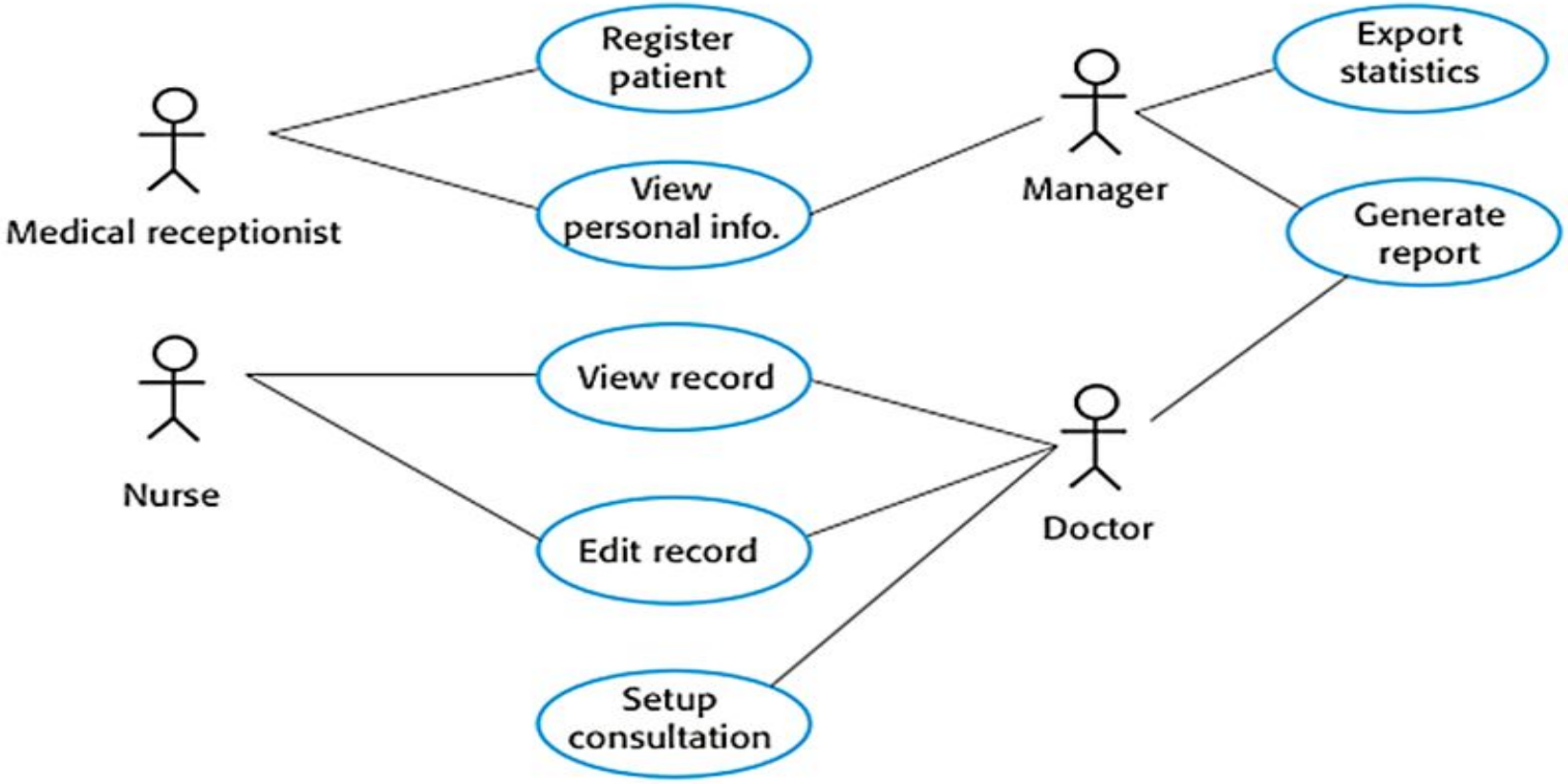
Other activities: Record may be consulted but not edited by other staff while information is being entered.

System state on completion: User is logged on. The patient record including medical history is entered in the database, a record is added to the system log showing the start and end time of the session and the nurse involved.

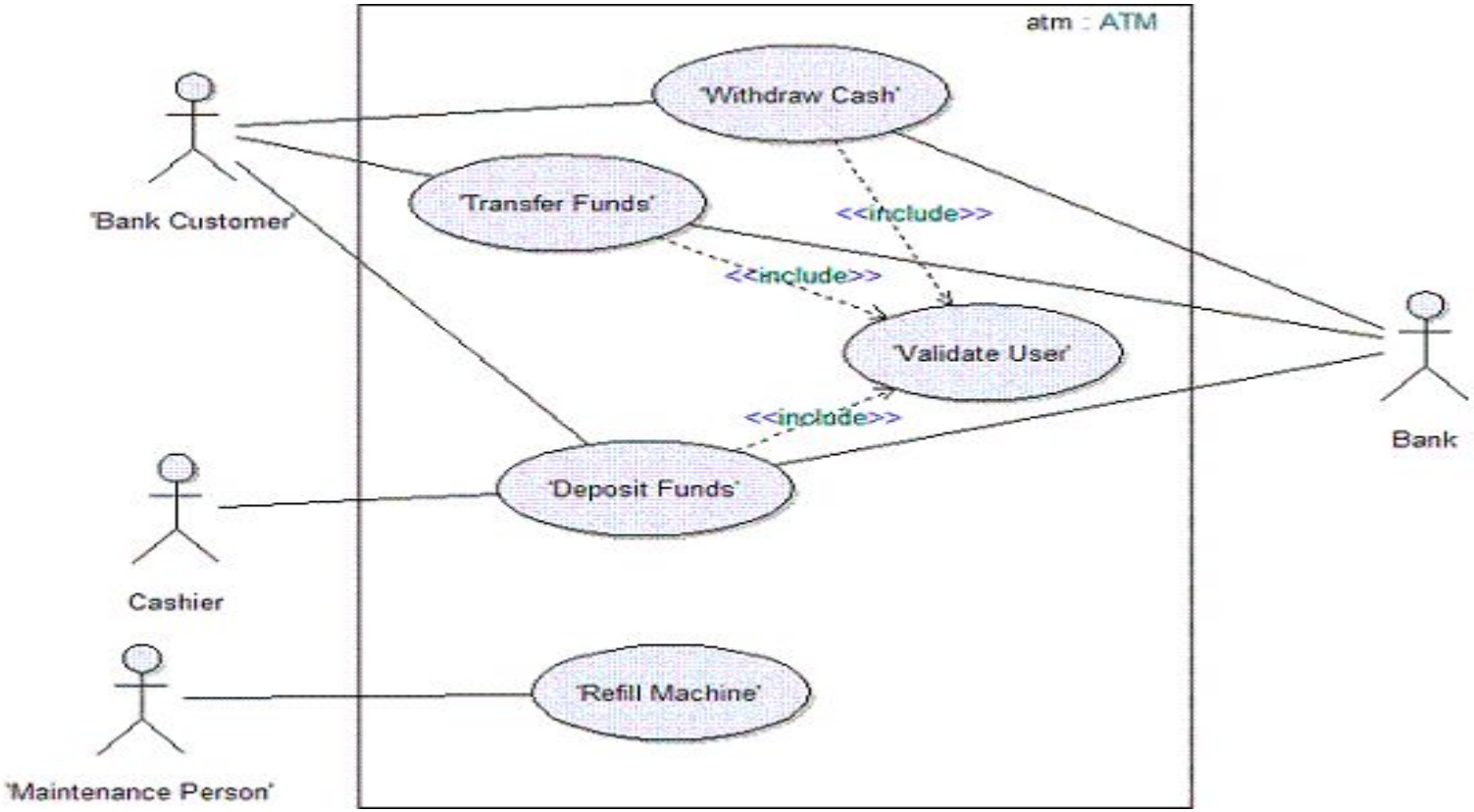
Use cases

- Use-cases are a scenario based technique in the UML which identify the actors in an interaction and which describe the interaction itself.
- A set of use cases should describe all possible interactions with the system.
- High-level graphical model supplemented by more detailed tabular description.
- Sequence diagrams may be used to add detail to usecases by showing the sequence of event processing in the system.

Use cases for the MHC-PMS



ATM


atm : ATM
'Withdraw Cash'
'Transfer Funds'
<<include>>
<<include>>
'Validate User'
<<include>>
'Deposit Funds'
'Refill Machine'
'Bank Customer'
Cashier
'Maintenance Person'
Bank

Ethnography

- A social scientist spends a considerable time observing and analysing how people actually work.
- People do not have to explain or articulate their work.
- Social and organizational factors of importance may be observed.
- Ethnographic studies have shown that work is usually richer and more complex than suggested by simple system models.

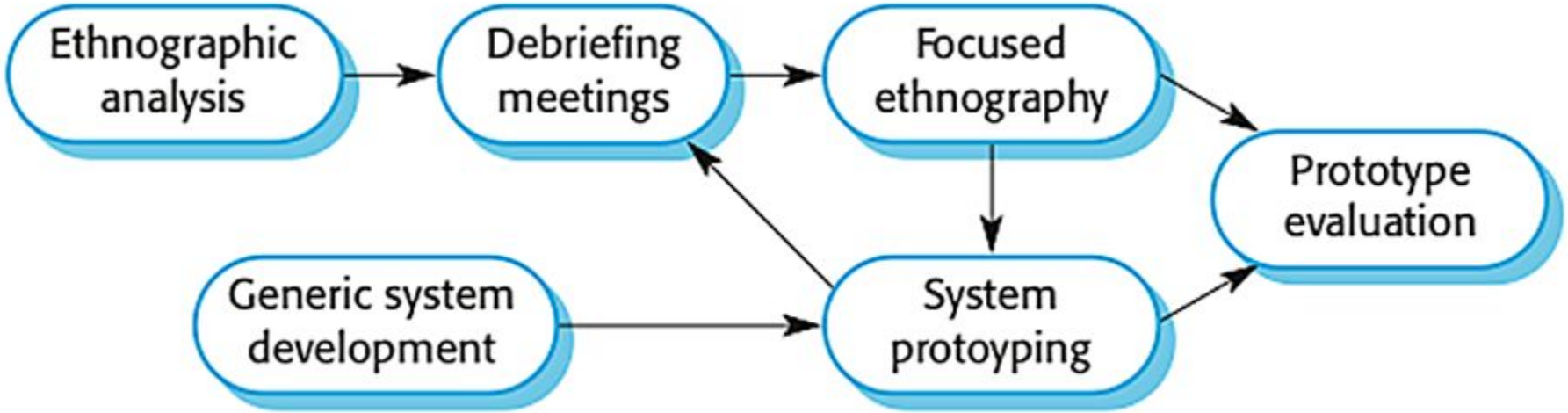
Scope of ethnography

- Requirements that are derived from the way that people actually work rather than the way in which process definitions suggest that they ought to work.
- Requirements that are derived from cooperation and awareness of other people's activities.
- Awareness of what other people are doing leads to changes in the ways in which we do things.
- Ethnography is effective for understanding existing processes but cannot identify new features that should be added to a system

Focused ethnography

- Developed in a project studying the air traffic control process
- Combines ethnography with prototyping
- Prototype development results in unanswered questions which focus the ethnographic analysis.
- The problem with ethnography is that it studies existing practices which may have some historical basis which is no longer relevant.

Ethnography and prototyping for requirements analysis



Requirements validation

- Concerned with demonstrating that the requirements define the system that the customer really wants.
- Requirements error costs are high so validation is very important
- Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.

Requirements checking

- *Validity*. Does the system provide the functions which best support the customer's needs?
- *Consistency*. Are there any requirements conflicts?
- *Completeness*. Are all functions required by the customer included?
- *Realism*. Can the requirements be implemented givenavailable budget and technology
- *Verifiability*. Can the requirements be checked?

Requirements validation techniques

- Requirements reviews
  - Systematic manual analysis of the requirements.
- Prototyping
  - Using an executable model of the system to check requirements.
- Test-case generation
  - Developing tests for requirements to check testability

Review checks

- Verifiability
- Is the requirement realistically testable?
- Comprehensibility
- Is the requirement properly understood?
- Traceability
- Is the origin of the requirement clearly stated?
- Adaptability
- Can the requirement be changed without a large impact on other requirements?

Requirements management

- Requirements management is the process of managing changing requirements during the requirements engineering process and system development.
- New requirements emerge as a system is being developed and after it has gone into use.
- need to keep track of individual requirements and maintain links between dependent requirements so that it is possible to assess the impact of requirements changes.
- need to establish a formal process for making change proposals and linking these to system requirements.

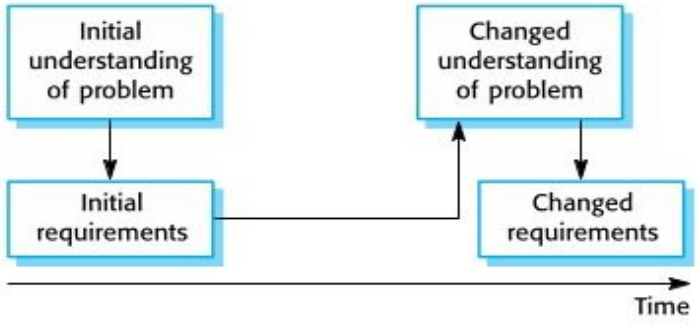
Changing requirements

- The business and technical environment of the system always changes after installation.
- New hardware may be introduced, it may be necessary to interface the system with other systems, business priorities may change (with consequent changes in the system support required), and new legislation and regulations may be introduced that the system must necessarily abide by.
- The people who pay for a system and the users of that system are rarely the same people.
- System customers impose requirements because of organizational and budgetary constraints. These may conflict with end-user requirements and, after delivery, new features may have to be added for user support if the system is to meet its goals.

Changing requirements

- Large systems usually have a diverse user community, with many users having different requirements and priorities that may be conflicting or contradictory.
- The final system requirements are inevitably a compromise between them and, with experience, it is often discovered that the balance of support given to different users has to be changed.

Requirements evolution



Requirements management planning

- Establishes the level of requirements management detail that is required.
- Requirements management decisions:

❖ Requirements identification Each requirement must be uniquely identified so that it can be cross-referenced with other requirements.

❖ A change management process This is the set of activities that assess the impact and cost of changes. I discuss this process

❖ Traceability policies These policies define the relationships between each requirement and between the requirements and the system design that should be recorded.

❖ Tool support Tools that may be used range from specialist requirements management systems to spreadsheets and simple database systems.

Requirements change management

- Deciding if a requirements change should be accepted

 Problem analysis and change specification

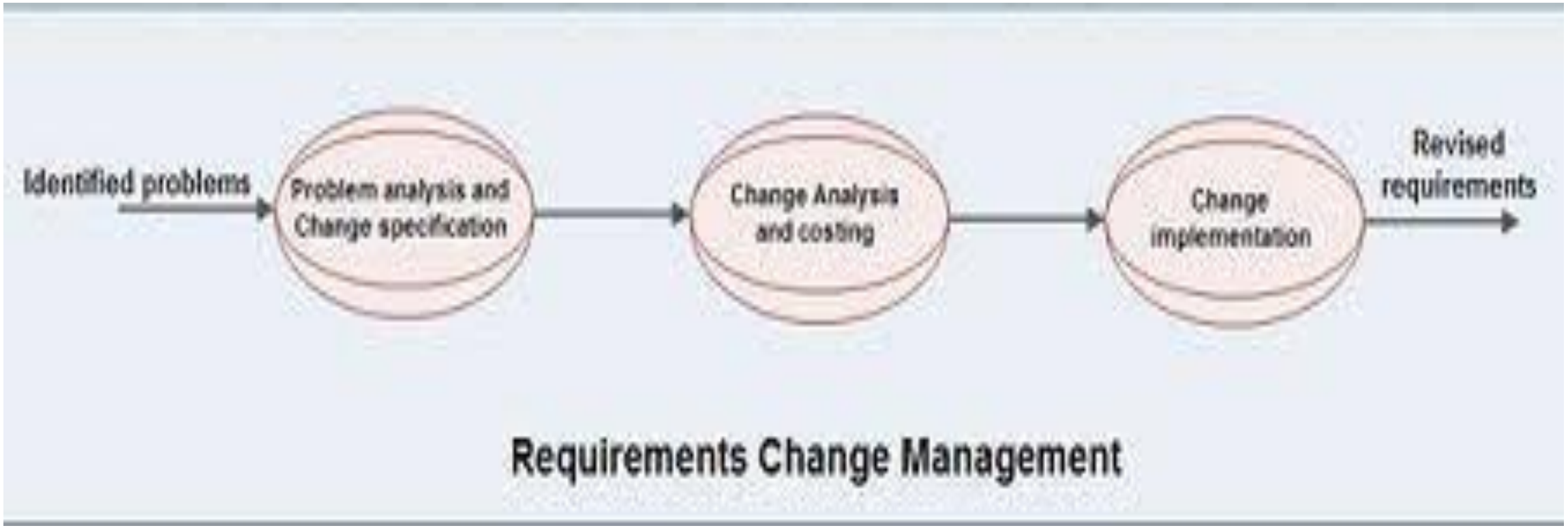
- During this stage, the problem or the change proposal is analyzed to check that it is valid. This analysis is fed back to the change requestor who may respond with a more specific requirements change proposal or decide to withdraw the request.

 Change analysis and costing

- The effect of the proposed change is assessed using traceability information and general knowledge of the system requirements. Once this analysis is completed, a decision is made whether or not to proceed with the requirements change.

 Change implementation

- The requirements document and, where necessary, the system design and implementation, are modified. Ideally, the document should be organized so that changes can be easily implemented.


Identified problems
Problem analysis and Change specification
Change Analysis and costing
Change implementation
Revised requirements
Requirements Change Management

CHAPTER 5

SYSTEM MODELING

CONTENTS

- **Context models**
- **Interaction models**
- **Structural models**
- **Behavioral models**
- **Model-driven engineering**

Objectives

- Introduce some types of system model that may be developed as part of the requirements engineering and system design processes.
- understand how graphical models can be used to represent software systems
- understand why different types of model are required and the fundamental system modeling perspectives of context, interaction, structure, and behavior.
- Use of UML diagrams
- be aware of the ideas underlying model-driven engineering, where a system is automatically generated from structural and behavioral models.

System perspectives represented by models

- An external perspective, where you model the context or environment of the system.
- An interaction perspective, where you model the interactions between a system and its environment, or between the components of a system.
- A structural perspective, where you model the organization of a system or the structure of the data that is processed by the system.
- A behavioral perspective, where you model the dynamic behavior of the system and how it responds to events

UML diagram types

- Activity diagrams, which show the activities involved in a process or in data processing .
- Use case diagrams, which show the interactions between a system and its environment.
- Sequence diagrams, which show interactions between actors and the system and between system components.
- Class diagrams, which show the object classes in the system and the associations between these classes.
- State diagrams, which show how the system reacts to internal and external events.

Use of graphical models

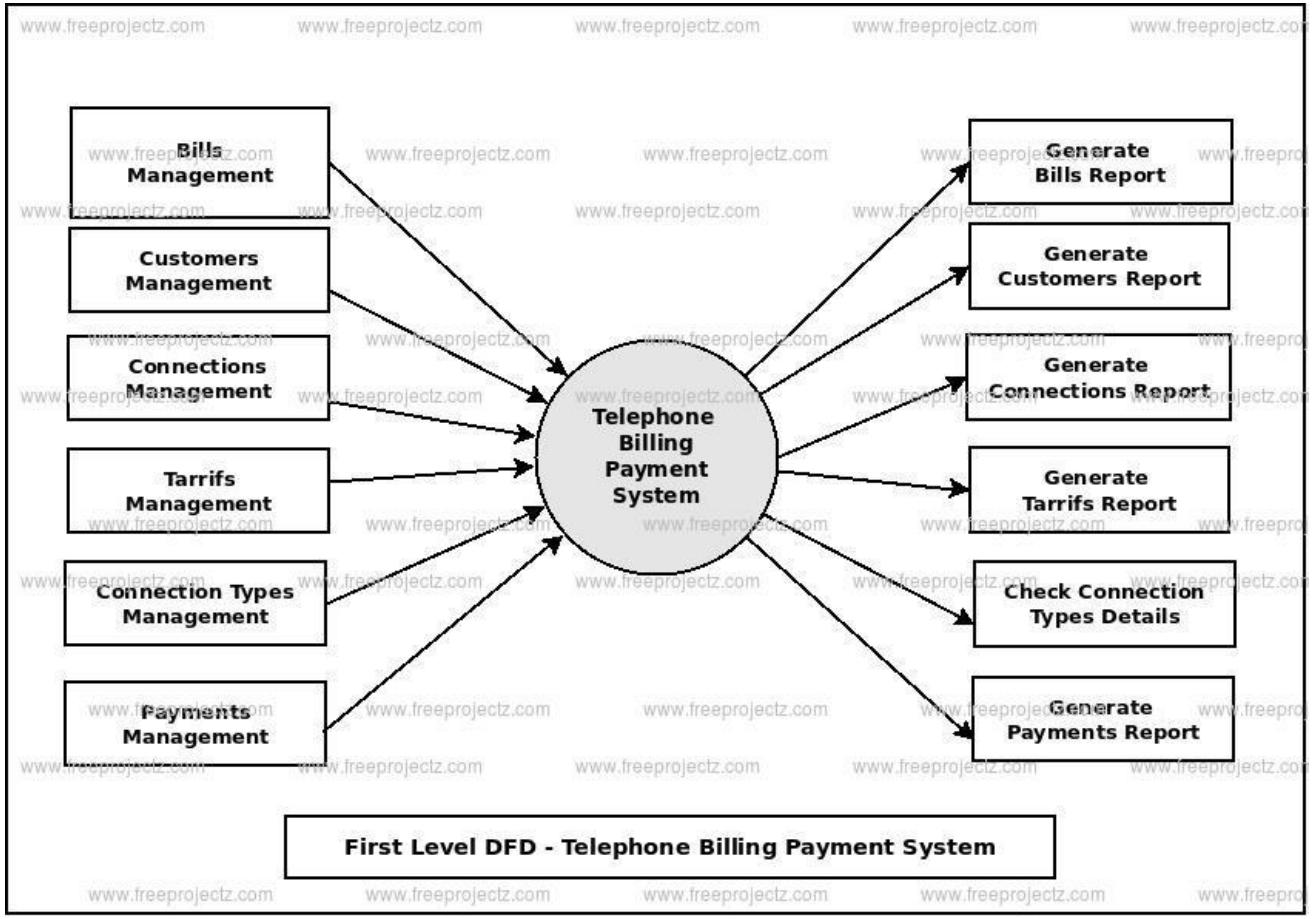
- As a means of facilitating discussion about an existing or proposed system
- As a way of documenting an existing system
- As a detailed system description that can be used to generate a system implementation

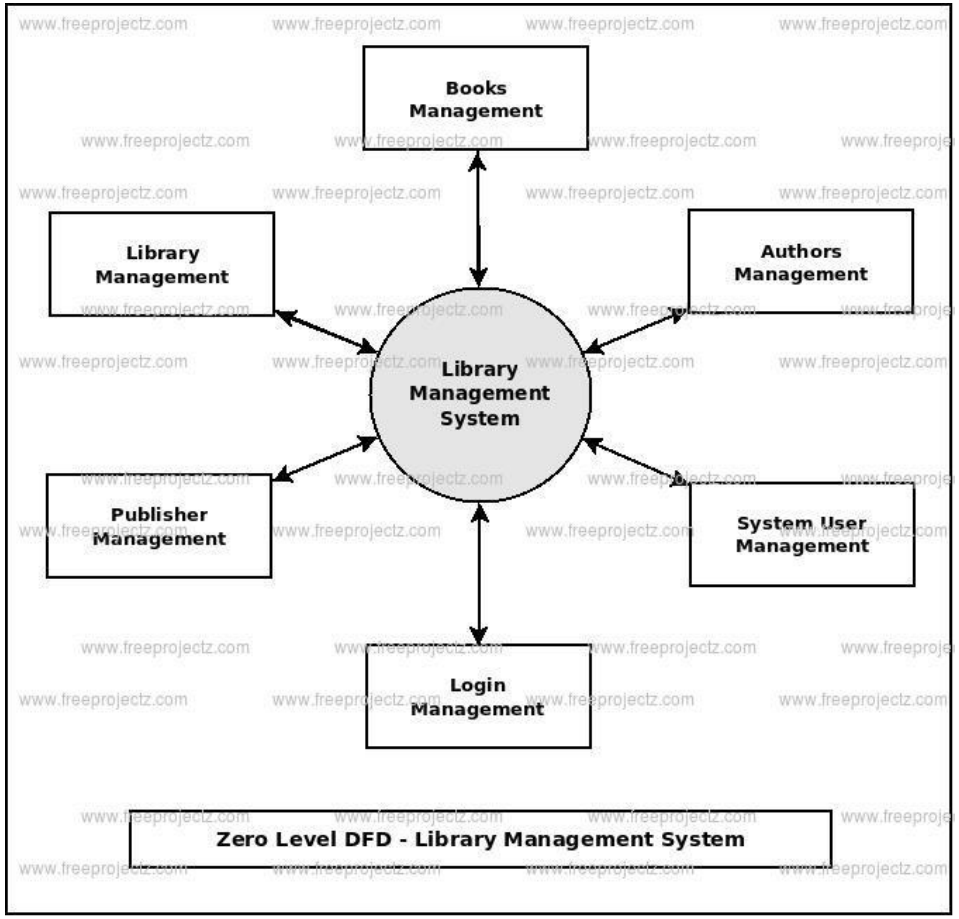
Context models

- Context models are used to illustrate the operational context of a system - they show what lies outside the system boundaries.
- Social and organisational concerns may affect the decision on where to position system boundaries.
- Architectural models show the system and its relationship with other systems.

System boundaries

- System boundaries are established to define what is inside and what is outside the system
- The position of the system boundary has a profound effect on the system requirements.
- Defining a system boundary is a political judgment


Bills Management
Customers Management
Connections Management
Tarrifs Management
Connection Types Management
Payments Management
Telephone Billing Payment System
Generate Bills Report
Generate Customers Report
Generate Connections Report
Generate Tarrifs Report
Check Connection Types Details
Generate Payments Report
First Level DFD - Telephone Billing Payment System


Books Management
Library Management
Authors Management
Library Management System
Publisher Management
System User Management
Login Management
Zero Level DFD - Library Management System

Interaction models

- Modeling user interaction is important as it helps to identify user requirements.
- Modeling system-to-system interaction highlights the communication problems that may arise.
- Modeling component interaction helps us understand if a proposed system structure is likely to deliver the required system performance and dependability.
- **Use case diagrams** and **sequence diagrams** may be used for interaction modeling.

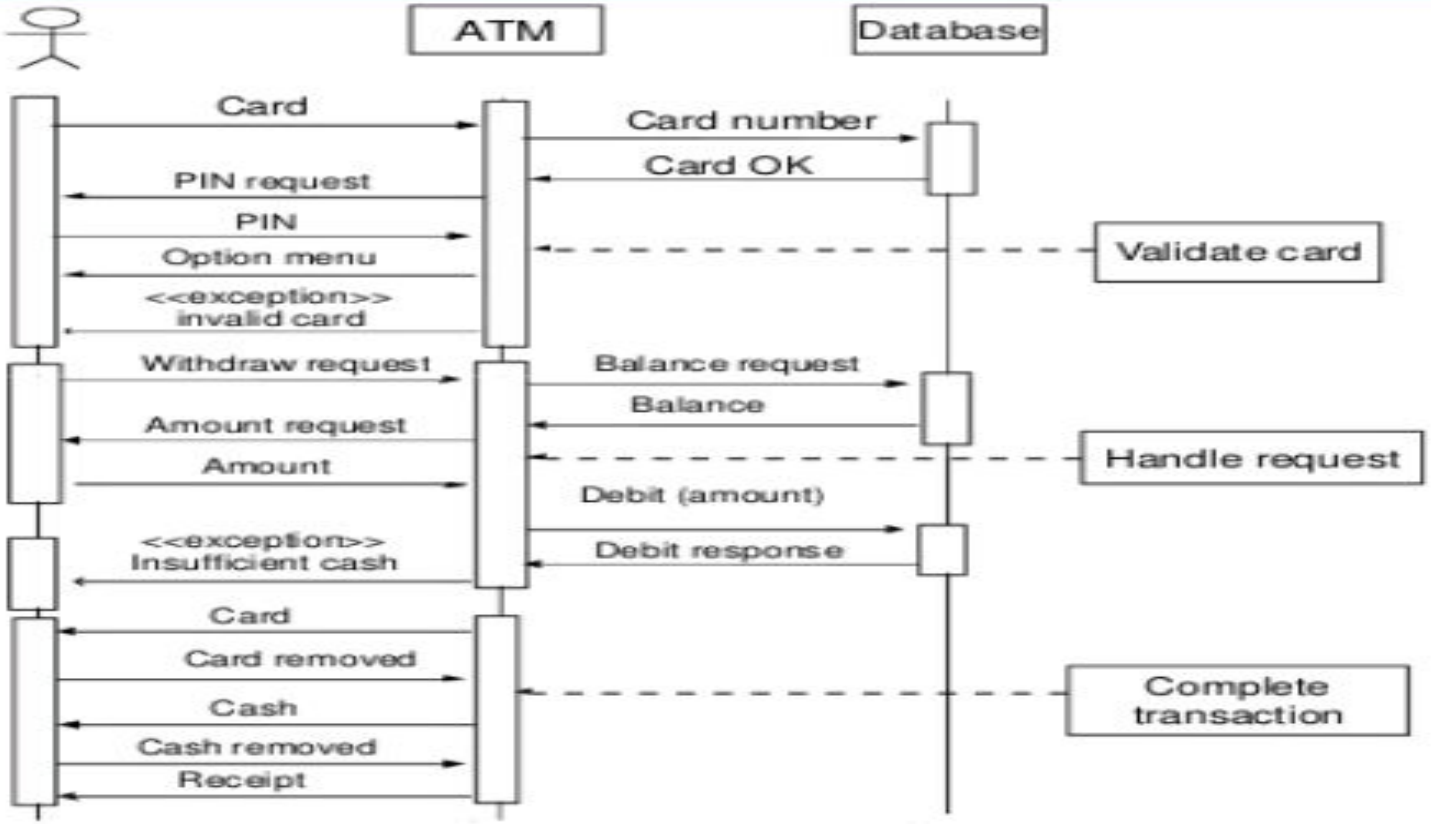
Use case modeling

- Use cases were developed originally to support requirements elicitation and now incorporated into the UML.
- Each use case represents a discrete task that involves external interaction with a system.
- Actors in a use case may be people or other systems.
- Represented diagramatically to provide an overview of the use case and in a more detailed textual form.

Sequence diagrams

- Sequence diagrams are part of the UML and are used to model the interactions between the actors and the objects within a system.
- A sequence diagram shows the sequence of interactions that take place during a particular use case or use case instance.
- The objects and actors involved are listed along the top of the diagram, with a dotted line drawn vertically from these.
- Interactions between objects are indicated by annotated arrows.

Sequence diagram of ATM withdrawal



Structural models

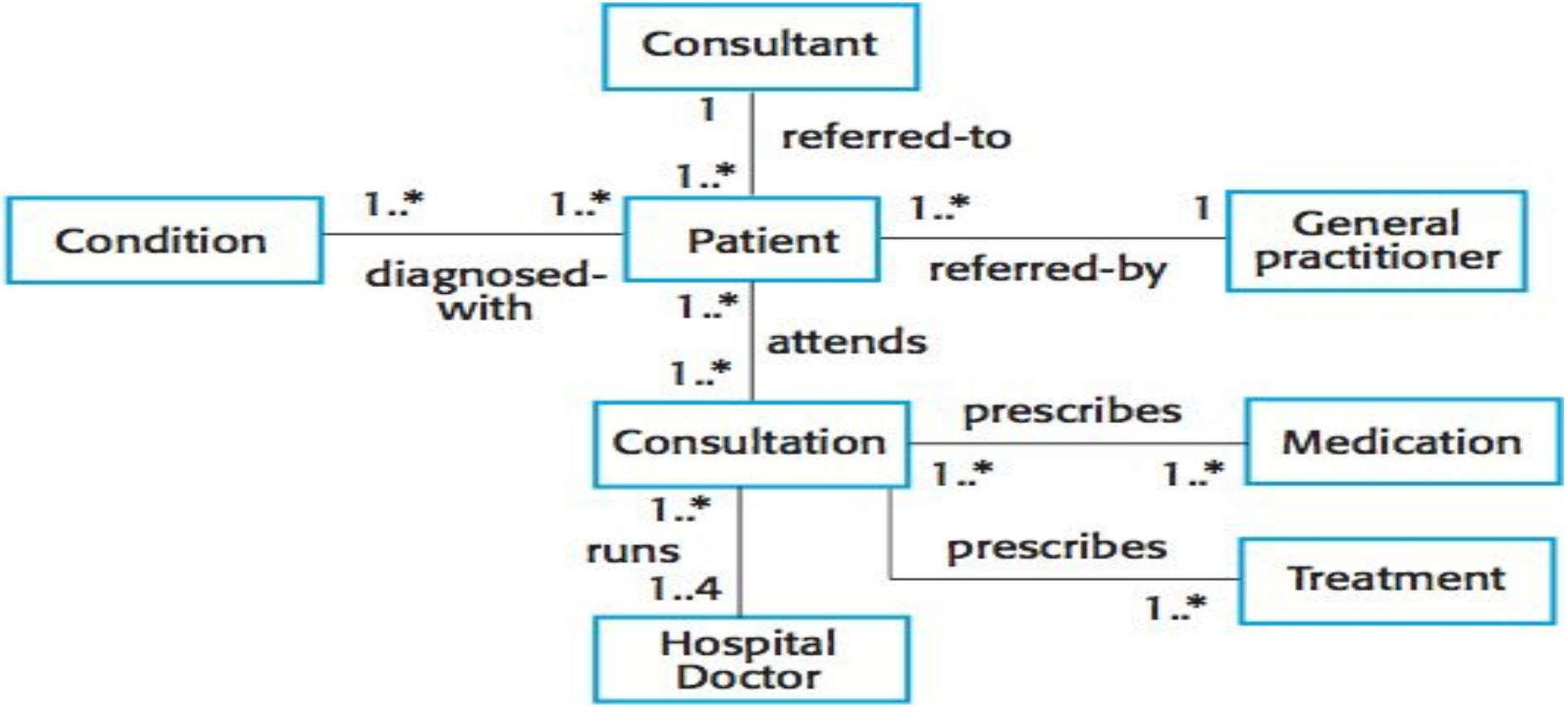
- Structural models of software display the organization of a system in terms of the components that make up that system and their relationships.
- Structural models may be static models, which show the structure of the system design, or dynamic models, which show the organization of the system when it is executing.
- create structural models of a system when discussing and designing the system architecture

Class diagrams

- Class diagrams are used when developing an object oriented system model to show the classes in a system and the associations between these classes.
- An object class can be thought of as a general definition of one kind of system object.
- An association is a link between classes that indicates that there is some relationship between these classes.
- When developing models during the early stages of the software engineering process, objects represent something in the real world, such as a patient, a prescription, doctor, etc.

Points wrt structural models

- Class diagram
- Association
- Generalization
- hierarchy


Consultant
1
referred-to
1..*
Condition
1..*
1..*
Patient
1..*
1
General practitioner
diagnosed-with
referred-by
1..*
attends
1..*
Consultation
prescribes
Medication
1..*
1..*
1..*
runs
1..4
prescribes
Treatment
1..*
Hospital Doctor

Behavioral models

- Behavioral models are models of the dynamic behavior of a system as it is executing. They show what happens or what is supposed to happen when a system responds to a stimulus from its environment
- Stimuli types:
- **Data** Some data arrives that has to be processed by the system.
- **Events** Some event happens that triggers system processing.

Data-driven modeling

- Many business systems are data-processing systems that are primarily driven by data. They are controlled by the data input to the system, with relatively little external event processing.
- Data-driven models show the sequence of actions involved in processing input data and generating an associated output

Event-driven modeling

- Real-time systems are often event-driven, with minimal data processing.
- Event-driven modeling shows how a system responds to external and internal events.
- It is based on the assumption that a system has a finite number of states and that events (stimuli) may cause a transition from one state to another.

State machine models

- These model the behaviour of the system in response to external and internal events.
- They show the system's responses to stimuli so are often used for modelling real-time systems.
- State machine models show system states as nodes and events as arcs between these nodes. When an event occurs, the system moves from one state to another.
- Statecharts are an integral part of the UML and are used to represent state machine models.

Model-driven engineering

- Model-driven engineering (MDE) is an approach to software development where models rather than programs are the principal outputs of the development process.
- The programs that execute on a hardware/software platform are then generated automatically from the models.
- Proponents of MDE argue that this raises the level of abstraction in software engineering so that engineers no longer have to be concerned with programming language details or the specifics of execution platforms

Usage of model-driven engineering

- Model-driven engineering is still at an early stage of development, and it is unclear whether or not it will have a significant effect on software engineering practice.
- **Pros**
- Allows systems to be considered at higher levels of abstraction
- Generating code automatically means that it is cheaper to adapt
- systems to new platforms.
- **Cons**
- Models for abstraction and not necessarily right for implementation.
- Savings from generating code may be outweighed by the costs of developing translators for new platforms.

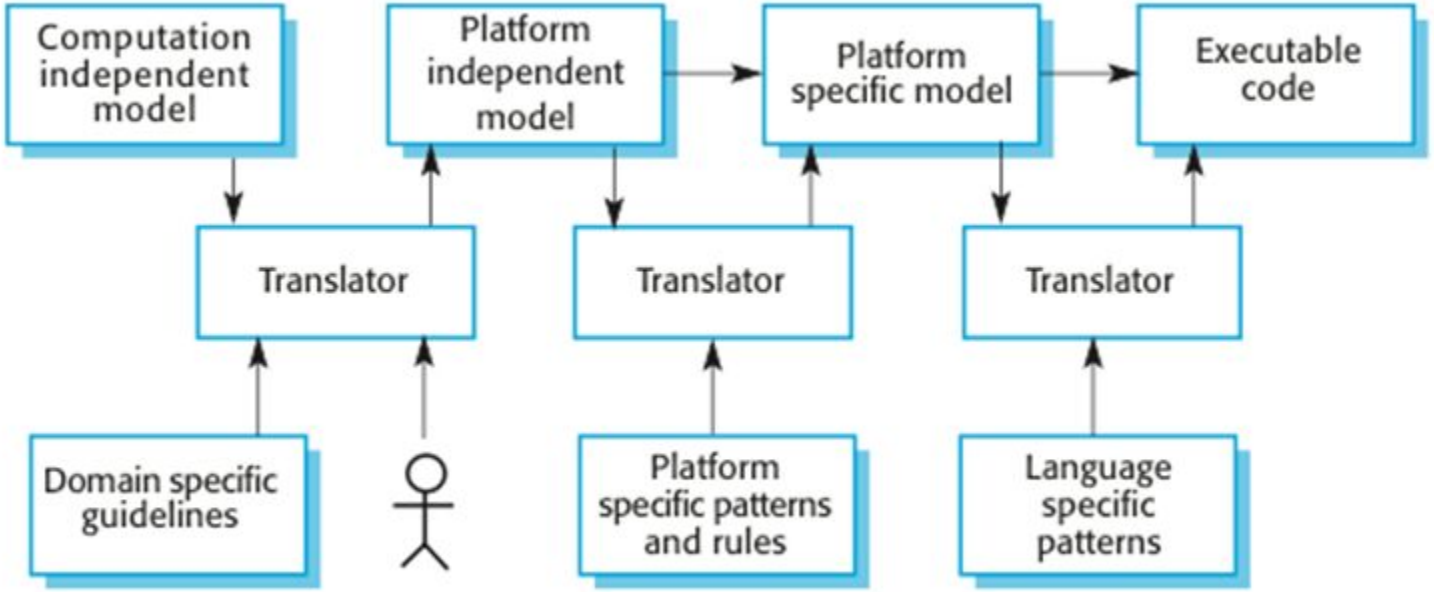
Model driven architecture

- Model-driven architecture (MDA) was the precursor of more general model-driven engineering
- MDA is a model-focused approach to software design and implementation that uses a subset of UML models to describe a system.
- Models at different levels of abstraction are created.
- From a high-level, platform independent model, it is possible, in principle, to generate a working program without manual intervention.

Types of model

- **A computation independent model (CIM)**
- These model the important domain abstractions used in a system. CIMs are sometimes called domain models.
- **A platform independent model (PIM)**
- These model the operation of the system without reference to its implementation. The PIM is usually described using UML models that show the static system structure and how it responds to external and internal events.
- **Platform specific models (PSM)**
- These are transformations of the platform-independent model with a separate PSM for each application platform. In principle, there may be layers of PSM, with each layer addingsome platform-specific detail.

MDA Transformations



Agile methods and MDA

- The developers of MDA claim that it is intended to support an iterative approach to development and so can be used within agile methods.
- If transformations can be completely automated and a complete program generated from a PIM, then, in principle, MDA could be used in an agile development process as no separate coding would be required.

Executable UML

- The fundamental notion behind model-driven engineering is that completely automated transformation of models to code should be possible.
- This is possible using a subset of UML 2, called Executable UML or xUML.

Features of executable UML

- To create an executable subset of UML,model types are:
- Domain models that identify the principal concerns in a system. They are defined using UML class diagrams and include objects,attributes and associations.
- Class models in which classes are defined, along with their attributes and operations.
- State models in which a state diagram is associated with each class and is used to describe the life cycle of the class.
- The dynamic behavior of the system may be specified declaratively using the object constraint language (OCL), or may be expressed using UML's action language.